

Management API

The backend now serves JSON at `/`; the legacy HTML auth/dashboard and JavaScript assets have been removed. The separate Next.js frontend is in `../react`. Apply migrations with `npm run migrate` before deploying. Migration 005 adds indexes for the new list queries, and migration 004 is now safe to rerun.

Authentication and roles

Bootstrap sudo access with `npm run seed:admin`, setting `SEED_ADMIN_EMAIL` and `SEED_ADMIN_PASSWORD` explicitly. The password must be 12–256 characters. No default credentials are supplied; existing accounts are left unchanged.

Existing `/v1/auth/signup`, `/v1/admin/login`, `/v1/admin/me`, `/v1/admin/logout`, and `/v1/admin/password` endpoints remain. Browser sessions use HTTP-only cookies with an eight-hour lifetime. `POST /v1/admin/password` requires `currentPassword` and `newPassword` (12–256 characters); success revokes all of that account's sessions.

The Next.js frontend uses a same-origin proxy and translates the session cookie path to `/`. Direct cookie-authenticated mutations must supply an Origin matching `PUBLIC_BASE_URL`. Service automation can use `Authorization: Bearer ADMIN_API_KEY` on supported admin endpoints. Never expose that key in a browser.

Tenant users can only access their own forms, submissions, analytics, and tenant details. Super and sudo admins can manage tenants. Only sudo admins can edit submissions or create super admins, as before.

List endpoints

Endpoint	Response collection	Filters
<code>GET /v1/admin/forms</code>	<code>forms</code>	<code>q</code> , <code>status=active disabled</code> , <code>tenantId</code> , <code>from</code> , <code>to</code> , <code>sort</code>
<code>GET /v1/admin/tenants</code>	<code>tenants</code>	<code>q</code> , <code>status=active inactive</code> , <code>tenantId</code> , <code>sort</code>
<code>GET /v1/admin/tenants/:tenantId</code>	<code>tenant</code> , <code>forms</code>	Form filters; tenant comes from the path
<code>GET /v1/admin/forms/:publicKey/submissions</code>	<code>submissions</code>	<code>q</code> (payload search), <code>status=accepted spam</code> , <code>from</code> , <code>to</code> , <code>sort=newest oldest</code>

All lists support `page` (default 1) and `limit` (default 20, maximum 100), and return `pagination: { page, limit, total, pages }`. Pages beyond the last page return an empty array and the correct total. Invalid pagination, UUIDs, date ranges, and statuses return 400. Dates are ISO-8601 timestamps including timezone; bounds are inclusive. Lists use SQL filtering and pagination with deterministic ID tie-breakers. Form dates filter the counted submissions, not the form creation date. Tenant totals are all-time retained totals. `sort` supports `newest`, `oldest`, `name`, and `most-used` on resource lists.

An active tenant has at least one active form; inactive includes tenants with no forms. Tenant summaries include form/active-form counts, retained and daily submissions, and all four limits. Use `GET /v1/admin/tenants?status=active`, then the tenant detail endpoint with `status=active` to retrieve its active forms. Tenant access never widens based on a supplied query tenant ID.

Analytics

`GET /v1/admin/analytics?tenantId=<uuid>&from=<ISO>&to=<ISO>` returns:

```
{
  "forms": 4,
  "activeForms": 3,
  "tenants": 2,
  "activeTenants": 1,
  "submissions": 50,
  "accepted": 48,
  "spam": 2,
  "mostUsedForms": [{ "publicKey": "...", "name": "Contact", "submissionCount": 40 }],
  "daily": [{ "day": "2026-09-25", "count": 12 }]
}
```

Top forms are limited to ten. Daily buckets are UTC and omit days without submissions. Submission counts exclude deleted records and are affected by retention cleanup. Dates filter submission totals/rankings, while form and tenant counts describe current resources. Tenant sessions always receive their own scope.

Tenant creation and password reset

`POST /v1/admin/tenants` with `{ "name": "Studio", "email": "owner@example.com", "password": "at-least-12-characters" }` atomically creates a tenant and its owner account. Requires super or sudo access. Returns 201 with `tenantId`, `name`, and `email`; duplicate emails return 409.

`POST /v1/admin/tenants/:tenantId/password` with `{ "email": "owner@example.com", "newPassword": "a-new-long-password" }` resets a matching tenant account and revokes all its sessions. It cannot reset super/sudo accounts and requires super/sudo access. Tenant users must use the current-password flow.

`PATCH /v1/admin/tenants/:tenantId` continues to accept all four positive-integer limits: `maxForms`, `maxOriginsPerForm`, `maxTotalSubmissions`, `maxDailySubmissions`.

The existing `/v1/admin/forms/summary` remains for compatibility. New clients should use the paginated form list. Submission list responses retain the `submissions` key and add pagination; the default page size is now 20 and maximum 100.

Automation, test submissions, and bulk replies

Run `npm run migrate` before using bulk replies (migration 008).

Each destination's `config.deliveryMode` is `automatic` or `manual`. Existing rules without a mode remain automatic; the editor defaults new rules to manual. Forms with no automatic rules only store incoming submissions. Paginated form records include `automaticRules` and `manualRules` counts.

Authenticated endpoints (tenant scoped; cookie mutations require same origin):

- `POST /v1/admin/forms/:publicKey/test`: { payload: { name: "Alex Morgan", email: "alex@example.com", message: "Let's build something together.", _website: "" }, send: false, requestId: "<uuid>" }. Validates against the saved schema without persistence or delivery. With `send: true`, creates an accepted submission and queues automatic rules; returns its `submissionId`. Reuse the request ID when retrying. Tests bypass public origin/bot checks, but enforce schema, workspace limits and admin rate limits. Use an address you control for real delivery.
- `GET /v1/admin/forms/:publicKey/delivery/:submissionId`: returns channel, status and attempt count for queued jobs. Provider acceptance does not guarantee inbox delivery.
- `POST /v1/admin/forms/:publicKey/replies`: { submissionIds: ["<uuid>"], destinationIds: ["<uuid>"], message: "Hello {{name}}", requestId: "<uuid>" }. Atomically queues up to 100 accepted, unexpired submissions across up to 20 active integrations from the same active form, including manual rules. Reusing the request ID returns the original queued count. Credentials and message content are snapshotted. Fixed destinations receive one message per selected submission; field destinations resolve each submitter's address.

The worker must be running with provider credentials configured. Webhook events include a `deliveryId` for receiver deduplication; manual replies use type `form.reply.created`. The legacy `id` remains the submission ID. Deduplicating only on the submission ID would suppress later replies.